Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Факультет электронно-информационных систем
Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №4

Специальность АС-68

Выполнил:

Студент 3-го курса

Группы АС-68

Ребенков А. Д.

Проверил:

А. А. Крощенко
Ст. преп. кафедры ИИТ,

«___»_____ 2026 г.

Брест 2026

Вариант 8

Цель работы: построить, обучить и оценить многослойный перцептрон (MLP) для решения задачи классификации

Задание:
Классификация семян пшеницы
• Seeds
• Задача: классифицировать семена на три сорта (3 класса).
• Архитектура:
  - входной слой;
  - один скрытый слой с 7 нейронами (ReLU);
  - выходной слой с 3 нейронами (Softmax).
• Эксперимент: увеличьте количество нейронов в скрытом слое до 14 и оцените, как это повлияло на точность.

Код программы:

```
import os
import pandas as pd
import torch
import torch.nn as nn
import torch.optim as optim
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import classification_report

torch.manual_seed(42)

script_dir = os.path.dirname(os.path.abspath(__file__))
df = pd.read_csv(os.path.join(script_dir, 'Seed_Data.csv'))
X = df.drop(columns=['target']).values
y = df['target'].values

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

X_train_t = torch.tensor(X_train_scaled, dtype=torch.float32)
y_train_t = torch.tensor(y_train, dtype=torch.long)
X_test_t = torch.tensor(X_test_scaled, dtype=torch.float32)
y_test_t = torch.tensor(y_test, dtype=torch.long)

class MLP(nn.Module):
    def __init__(self, hidden_dim):
        super().__init__()
        self.fc1 = nn.Linear(7, hidden_dim)
        self.relu = nn.ReLU()
        self.fc2 = nn.Linear(hidden_dim, 3)

    def forward(self, x):
        return self.fc2(self.relu(self.fc1(x)))

def run_experiment(hidden_dim):
```

```
    model = MLP(hidden_dim)
    criterion = nn.CrossEntropyLoss()
    optimizer = optim.Adam(model.parameters(), lr=0.01)

    for _ in range(200):
        model.train()
        optimizer.zero_grad()
        loss = criterion(model(X_train_t), y_train_t)
        loss.backward()
        optimizer.step()

    model.eval()
    with torch.no_grad():
        preds = torch.argmax(model(X_test_t), dim=1)

    print(f"=== MLP with {hidden_dim} hidden neurons ===")
    print(classification_report(y_test_t.numpy(), preds.numpy(), digits=4))

run_experiment(7)
run_experiment(14)
```

Результат работы программы:

```
=== MLP with 7 hidden neurons ===
              precision    recall  f1-score   support

           0     0.8889    0.5714    0.6957        14
           1     0.8125    0.9286    0.8667        14
           2     0.8235    1.0000    0.9032        14

    accuracy                         0.8333        42
   macro avg     0.8416    0.8333    0.8218        42
weighted avg     0.8416    0.8333    0.8218        42

=== MLP with 14 hidden neurons ===
              precision    recall  f1-score   support

           0     1.0000    0.6429    0.7826        14
           1     0.8750    1.0000    0.9333        14
           2     0.8235    1.0000    0.9032        14

    accuracy                         0.8810        42
   macro avg     0.8995    0.8810    0.8731        42
weighted avg     0.8995    0.8810    0.8731        42
```

Вывод: построил, обучил и оценил многослойный перцептрон (MLP) для решения задачи классификации